

Parcial 1 Proyectos (Sistema de Analítica Académica)

Maryom Alejandra Vicente González

Carné: 202202918

Proyectos de Computación AIE [0980]

Facultad de Ingeniería

Universidad de San Carlos de Guatemala

Resumen—Se desarrolló un Sistema de Analítica Académica en GNU Octave y Python para registrar estudiantes, calcular notas, consultar historiales y generar gráficas de rendimiento. Los datos se almacenan en `notas.txt` y PostgreSQL.

I. REPOSITORIO

El código desarrollado durante la actividad se encuentra disponible en el siguiente repositorio privado de GitHub:

<https://github.com/3050321780117-coder/Repopruebamaryom>

II. INTRODUCCIÓN

Se desarrolló un sistema en GNU Octave y Python para registrar, calcular y analizar notas de estudiantes, utilizando `notas.txt` y PostgreSQL para almacenar los datos.

III. DESCRIPCIÓN DEL PROBLEMA

El sistema registra el carné, nombre y calificaciones de cada estudiante. La nota final se calcula con las ponderaciones de Tareas (25 %), Parcial 1 (16 %), Parcial 2 (17 %), Parcial 3 (17 %) y Examen Final (25 %). El curso se aprueba con una nota mínima de 61 puntos.

IV. CÁLCULO DE LA NOTA FINAL

La nota final se obtiene aplicando las ponderaciones establecidas:

$$N = 0.25T + 0.16P_1 + 0.17P_2 + 0.17P_3 + 0.25EF \quad (1)$$

El resultado se redondea según su parte decimal y se considera **Aprobado** con una nota mayor o igual a 61.

V. ALGORITMO

1. Iniciar el programa.
2. Conectarse con PostgreSQL.
3. Mostrar el menú principal.
4. Leer y validar la opción seleccionada.
5. Si se elige ingreso de datos:
 - Solicitar carné y nombre.
 - Solicitar las cinco calificaciones.
 - Validar los valores.
 - Calcular la nota ponderada.
 - Aplicar el redondeo.
 - Determinar Aprobado o Reprobado.

- Mostrar el resultado.
- Guardar en `notas.txt`.
- Guardar en PostgreSQL.

6. Si se elige historial:

- Consultar los registros.
- Mostrar los datos en consola.

7. Si se elige analítica visual:

- Obtener los registros.
- Generar gráfica de rendimiento individual.
- Generar gráfica de aprobados y reprobados.
- Calcular promedios por rubro.
- Generar gráfica de desempeño por rubro.

8. Si se elige salir:

- Cerrar PostgreSQL.
- Finalizar el programa.

VI. DIAGRAMA DE FLUJO

El diagrama representa la secuencia general del sistema.

Figura 1. Diagrama de flujo general del sistema de analítica académica.

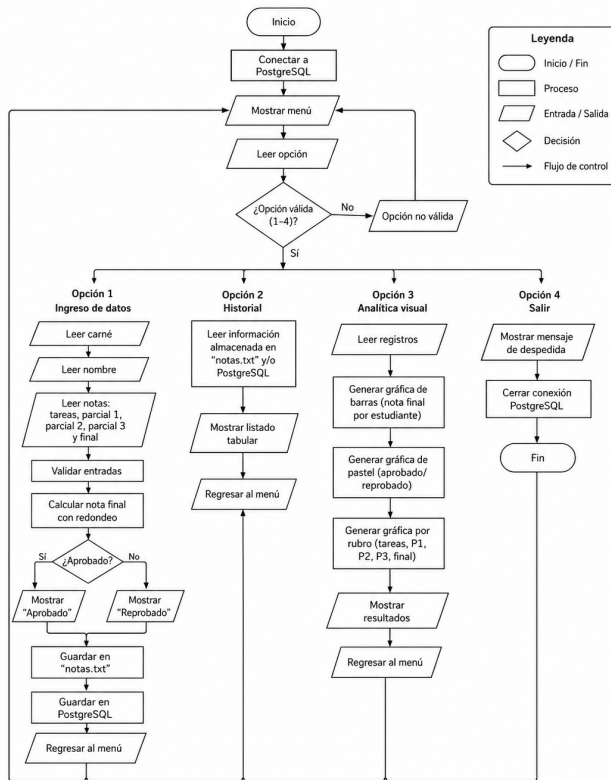


Figura 1. Diagrama de flujo del sistema.

VII. IMPLEMENTACIÓN

El programa fue implementado en GNU Octave y Python utilizando condicionales, ciclos, validaciones y funciones.

VII-A. Modularización

La modularización consiste en dividir el programa en funciones pequeñas, donde cada función realiza una tarea específica.

Entre las funciones utilizadas se encuentran:

- Validación de notas.
- Cálculo de la nota final.
- Aplicación del redondeo.
- Determinación del estado.
- Escritura del archivo de texto.
- Almacenamiento en PostgreSQL.
- Consulta del historial.
- Generación de gráficas.

VIII. MENÚ PRINCIPAL

El sistema presenta cuatro opciones principales:

1. Ingreso de datos.
2. Historial.
3. Analítica visual.

4. Salir.

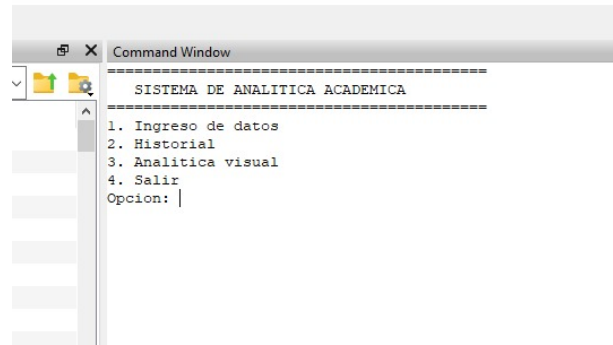


Figura 2. Menú principal del programa.

IX. INGRESO DE DATOS

La primera opción solicita los datos académicos del estudiante.

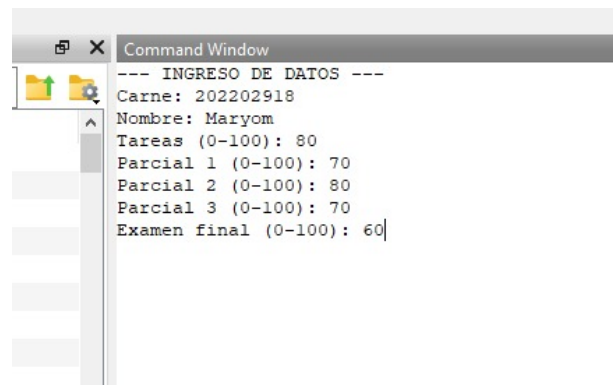


Figura 3. Ingreso de datos del estudiante.

IX-A. Validación de Datos

Si el usuario ingresa letras o valores no permitidos, el programa solicita nuevamente la información.

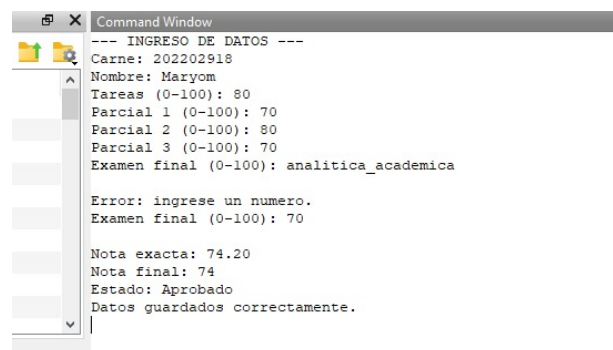


Figura 4. Validación de una entrada incorrecta.

IX-B. Resultado

Después de realizar el cálculo se muestra la nota exacta, nota final y estado académico.

```
Nota exacta: 74.20
Nota final: 74
Estado: Aprobado
Datos guardados correctamente.
```

Figura 5. Resultado del cálculo académico.

X. ALMACENAMIENTO EN ARCHIVO DE TEXTO

La información se almacena localmente en:

notas.txt

El archivo se crea si no existe y los nuevos registros se agregan al final.

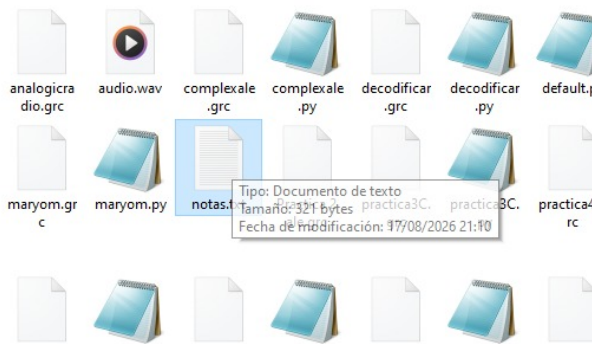


Figura 6. Información almacenada en notas.txt.

XI. BASE DE DATOS POSTGRESQL

La información también se almacena en PostgreSQL.

```
CREATE TABLE notas_estudiantes (
  id SERIAL PRIMARY KEY,
  carnet VARCHAR(20) NOT NULL,
  nombre VARCHAR(100) NOT NULL,
  tareas DOUBLE PRECISION NOT NULL,
  p1 DOUBLE PRECISION NOT NULL,
  p2 DOUBLE PRECISION NOT NULL,
  p3 DOUBLE PRECISION NOT NULL,
  examen_final DOUBLE PRECISION NOT NULL,
  nota_exacta DOUBLE PRECISION NOT NULL,
  nota_final INTEGER NOT NULL,
  estado VARCHAR(20) NOT NULL
);
```

id	carnet	nombre	tareas	p1	p2	p3	examen_final	nota_exacta	nota_final	estado
1	202202918	Maryom	68	72	73	74	74.20	74.20	74	Aprobado
2	202201122	Derek	72	73	74	75	75.00	75.00	75	Aprobado
3	202100224	Ana	73	74	75	76	76.00	76.00	76	Aprobado
4	202200000	Luis	50	51	52	53	53.00	53.00	53	Reprobado
5	202100328	Kevin	56	57	58	59	59.00	59.00	59	Reprobado
6	202202918	Maryom	74	75	76	77	77.00	77.00	77	Aprobado

Figura 7. Registros almacenados en PostgreSQL.

XII. HISTORIAL DE DATOS

La segunda opción muestra los registros almacenados en forma tabular.

CARNE	NOMBRE	NOTA	ESTADO
202202918	Maryom	68	Aprobado
202201122	Derek	72	Aprobado
202100224	Ana	73	Aprobado
202200000	Luis	50	Reprobado
202100328	Kevin	56	Reprobado
202202918	Maryom	74	Aprobado

Figura 8. Historial de estudiantes.

XIII. MÓDULO DE ANALÍTICA VISUAL

El sistema genera tres gráficas para analizar el rendimiento del grupo.

XIII-A. Rendimiento Individual

La gráfica de barras compara la nota final de cada estudiante.

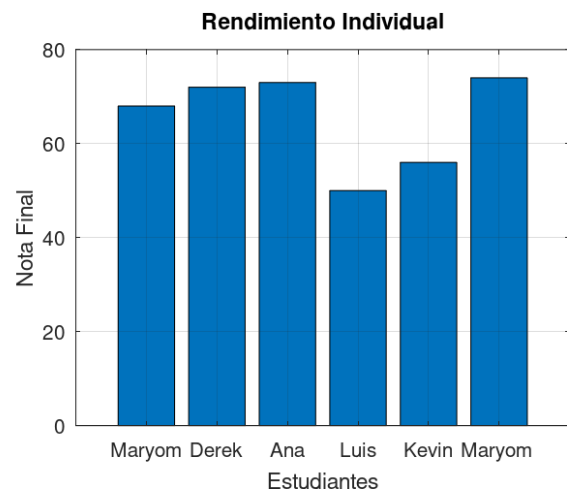


Figura 9. Análisis de rendimiento individual.

XIII-B. Índice de Aprobación

La gráfica de pastel muestra el porcentaje de estudiantes aprobados y reprobados.



Figura 10. Índice de aprobación.

XIII-C. Desempeño por Rubro

La gráfica muestra el promedio obtenido por el grupo en cada evaluación.

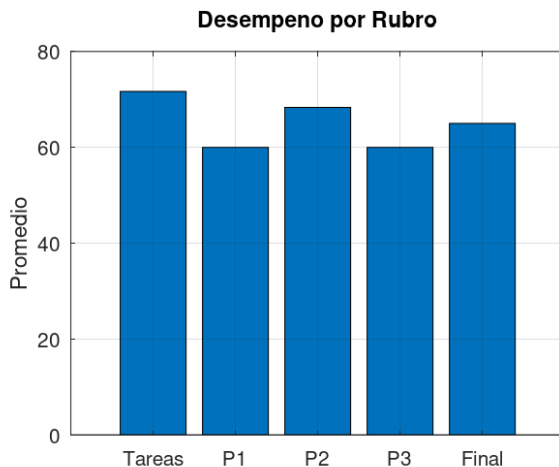


Figura 11. Desempeño promedio por rubro.

XIV. VALIDACIÓN Y MANEJO DE ERRORES

El programa valida las opciones del menú y las calificaciones ingresadas.

La información se almacena primero en el archivo `notas.txt`. Luego se intenta guardar en PostgreSQL. Si ocurre un error, el registro permanece en el archivo de texto y el usuario recibe una notificación.

XV. IMPLEMENTACIÓN EN GNU OCTAVE

La versión de GNU Octave utiliza el paquete `database` y varias funciones separadas en archivos `.m`.

XVI. CÓDIGO EN GNU OCTAVE

XVI-A. Programa Principal

```
1 clc;
2 clear;
3 pkg load database
4
5 % Configuración
6 dbname = 'test';
7 host = 'localhost';
8 port = '5432';
9 usuario = 'postgres';
10 password = 'rockyto';
11 archivo_txt = 'notas.txt';
12
13 % Conexión PostgreSQL
14 try
15     conn = pg_connect(setdbopts( ...
16         'dbname', dbname, ...
17         'host', host, ...
18         'port', port, ...
19         'user', usuario, ...
20         'password', password));
21     conexion_activa = true;
22
23 catch
24     disp('Error de conexión con PostgreSQL.');
```

Figura 12. Programa principal desarrollado en GNU Octave.

```

58     clc;
59
60     disp('--- INGRESO DE DATOS ---');
61
62     carnet = input('Carnet: ', 's');
63     nombre = input('Nombre: ', 's');
64
65     tareas = leer_notas('Tareas (0-100): ');
66     p1 = leer_notas('Parcial 1 (0-100): ');
67     p2 = leer_notas('Parcial 2 (0-100): ');
68     p3 = leer_notas('Parcial 3 (0-100): ');
69     examen_final = leer_notas('Examen final (0-100): ');
70
71     % Calculo
72     nota_exacta = calcular_nota( ...
73         tareas, p1, p2, p3, examen_final);
74
75     % Redondeo
76     nota_final = redondear_nota(nota_exacta);
77
78     % Estado
79     estado = determinar_estado(nota_final);
80
81     % Resultado
82     fprintf('\nNota exacta: %.2f\n', nota_exacta);
83     fprintf('Nota final: %d\n', nota_final);
84     fprintf('Estado: %s\n', estado);
85
86     % Guardar primero en TXT
87     guardar_txt( ...
88         archivo_txt, carnet, nombre, ...
89         tareas, p1, p2, p3, examen_final, ...
90         nota_exacta, nota_final, estado);
91
92     % Guardar PostgreSQL
93     try
94
95         guardar_postgresql( ...
96             conn, carnet, nombre, ...
97             tareas, p1, p2, p3, examen_final, ...
98             nota_exacta, nota_final, estado);
99
100        disp('Datos guardados correctamente.');

```

Figura 13. Programa principal desarrollado en GNU Octave.

```

115 % Opcion 2
116 elseif opcion == 2
117     mostrar_historial(conn, archivo_txt);
118     pause;
119
120 % Opcion 3
121 elseif opcion == 3
122     analitica_visual(conn);
123     pause;
124
125 % Opcion 4
126 elseif opcion == 4
127     disp('Gracias por utilizar el programa.');

```

Figura 14. Programa principal desarrollado en GNU Octave.

XVI-B. Validación de Notas

```

1 function nota = leer_nota(mensaje)
2
3 while true
4
5     entrada = input(mensaje, 's');
6     nota = str2double(entrada);
7
8     if isnan(nota)
9
10        disp('Error: ingrese un numero.');

```

Figura 15. Función para validar las calificaciones ingresadas.

XVI-C. Cálculo de la Nota

```

1 function nota = calcular_nota(tareas, p1, p2, p3, examen_final)
2
3     nota = ...
4         tareas * 0.25 + ...
5         p1 * 0.16 + ...
6         p2 * 0.17 + ...
7         p3 * 0.17 + ...
8         examen_final * 0.25;
9
10 endfunction
11
12

```

Figura 16. Función para realizar el cálculo ponderado de la nota.

XVI-D. Redondeo de la Nota

```

1 function nota_final = redondear_nota(nota_exakta)
2
3     entero = floor(nota_exakta);
4     decimal = nota_exakta - entero;
5
6     if decimal >= 0.5
7
8         nota_final = entero + 1;
9
10    else
11
12        nota_final = entero;
13
14    endif
15
16 endfunction
17
18

```

Figura 17. Función para aplicar la regla de redondeo.

XVI-E. Estado Académico

```
1 function estado = determinar_estado(nota_final)
2
3     if nota_final >= 61
4
5         estado = 'Aprobado';
6
7     else
8
9         estado = 'Reprobado';
10
11     endif
12
13 endfunction
```

Figura 18. Función para determinar el estado Aprobado o Reprobado.

XVI-F. Almacenamiento en Archivo de Texto

```

1 function guardar_txt( ...
2     archivo_txt, carnet, nombre, ...
3     tareas, p1, p2, p3, examen_final, ...
4     nota_exacta, nota_final, estado)
5
6     archivo = fopen(archivo_txt, 'a');
7
8     if archivo == -1
9         error('No se pudo abrir notas.txt');
10    endif
11
12    fprintf(archivo, ...
13        '%s%s%.2f%.2f%.2f%.2f%.2f%.2f%.2f%.2f\n', ...
14        carnet, nombre, ...
15        tareas, p1, p2, p3, examen_final, ...
16        nota_exacta, nota_final, estado);
17
18    fclose(archivo);
19
20 endfunction

```

Figura 19. Función para almacenar los registros en notas.txt.

XVI-G. Almacenamiento en PostgreSQL

```

1 function guardar_postgresql( ...
2     conn, carnet, nombre, ...
3     tareas, p1, p2, p3, examen_final, ...
4     nota_exacta, nota_final, estado)
5
6     carnet_sql = sprintf(carnet, "%s", "");
7     nombre_sql = sprintf(nombre, "%s", "");
8     estado_sql = sprintf(estado, "%s", "");
9
10    consulta = sprintf( ...
11        ["INSERT INTO notas_estudiantes " ...
12         "(carnet,nombre,tareas,p1,p2,p3,examen_final, ...
13          \"nota_exacta,nota_final,estado) ...
14          \"VALUES ('%s','%s',%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%d,'%s')\";], ...
15        carnet_sql, nombre_sql, ...
16        tareas, p1, p2, p3, examen_final, ...
17        nota_exacta, nota_final, estado_sql);
18
19    pq_exec_params(conn, consulta);
20
21 endfunction

```

Figura 20. Función para almacenar los registros en PostgreSQL.

```

1 function mostrar_historial(conn, archivo_txt)
2
3     clc;
4
5     disp('=====');
6     disp('          HISTORIAL');
7     disp('=====');
8
9     try
10
11         datos = pq_exec_params(conn, ...
12             ['SELECT carnet,nombre,nota_final,estado " ...
13              "FROM notas_estudiantes ORDER BY id:"]);
14
15         if isempty(datos.data)
16
17             disp('No hay estudiantes registrados.'):
18
19         else
20
21             fprintf('\n%-15s %-25s %-10s %-15s\n', ...
22                 'CARNE', 'NOMBRE', 'NOTA', 'ESTADO');
23
24             disp('-----');
25
26             for i = 1:rows(datos.data)
27
28                 fprintf('%-15s %-25s %-10.0f %-15s\n', ...
29                     datos.data(i,1), ...
30                     datos.data(i,2), ...
31                     datos.data(i,3), ...
32                     datos.data(i,4));
33
34             endfor
35
36         endif
37
38     catch
39
40         disp('Error al consultar PostgreSQL.'):
41         disp('Mostrando notas.txt.'):
42
43         if exist(archivo_txt, 'file')
44
45             archivo = fopen(archivo_txt, 'r');
46
47             while ~feof(archivo)
48
49                 linea = fgetl(archivo);
50
51                 if ischar(linea)
52                     disp(linea);
53                 endif
54
55             endwhile
56
57             fclose(archivo);
58
59         else
60
61             disp('No existe notas.txt.'):
62
63         endif
64
65     end_try_catch
66
67 endfunction
68

```

Figura 21. Función para consultar y mostrar el historial de estudiantes.

```

1 function analitica_visual(conn)
2
3     clc;
4
5     disp('=====');
6     disp('          ANALITICA VISUAL');
7     disp('=====');
8
9     datos = pq_exec_params(conn, ...
10         ['SELECT nombre,tareas,p1,p2,p3,examen_final," ...
11          "nota_final,estado " ...
12          "FROM notas_estudiantes ORDER BY id:"]);
13
14     if isempty(datos.data)
15
16         disp('No hay datos para generar graficas.'):
17         return;
18
19     endif
20
21     cantidad = rows(datos.data);
22
23     nombres = cell(cantidad,1);
24     tareas = zeros(cantidad,1);
25     p1 = zeros(cantidad,1);
26     p2 = zeros(cantidad,1);
27     p3 = zeros(cantidad,1);
28     finales = zeros(cantidad,1);
29     notas = zeros(cantidad,1);
30     estados = cell(cantidad,1);
31
32     % Obtener datos
33     for i = 1:cantidad
34
35         nombres(i) = datos.data(i,1);
36         tareas(i) = datos.data(i,2);
37         p1(i) = datos.data(i,3);
38         p2(i) = datos.data(i,4);
39         p3(i) = datos.data(i,5);
40         finales(i) = datos.data(i,6);
41         notas(i) = datos.data(i,7);
42         estados(i) = datos.data(i,8);
43
44     endfor
45
46     % Grafica 1
47     figure(1);
48     clf;
49
50     bar(notas);
51
52     title('Rendimiento Individual');
53     xlabel('Estudiantes');
54     ylabel('Nota Final');
55
56     set(gca, ...
57         'xtick', 1:cantidad, ...
58         'xticklabel', nombres);
59
60     grid on;
61
62

```

Figura 22. Función para generar las visualizaciones de analítica académica.

```

62 grid on;
63
64
65 % Grafica 2
66 aprobados = sum(strcmp(estados, 'Aprobado'));
67 reprobados = sum(strcmp(estados, 'Reprobado'));
68
69 figure(2);
70 clf;
71
72 pie([aprobados, reprobados]);
73
74 legend('Aprobados', 'Reprobados');
75
76 title('Indice de Aprobacion');
77
78
79 % Grafica 3
80 promedios = [ ...
81     mean(tareas), ...
82     mean(p1), ...
83     mean(p2), ...
84     mean(p3), ...
85     mean(finales)];
86
87 figure(3);
88 clf;
89
90 bar(promedios);
91
92 title('Desempeno por Rubro');
93 ylabel('Promedio');
94
95 set(gca, ...
96     'xtick', 1:5, ...
97     'xticklabel', ...
98     {'Tareas', 'P1', 'P2', 'P3', 'Final'});
99
100 grid on;
101
102 disp('Graficas generadas correctamente.');
```

Función para generar las visualizaciones de analítica académica.

XVII. IMPLEMENTACIÓN EN PYTHON

La versión en Python utiliza psycopg para PostgreSQL y matplotlib para las gráficas.

XVIII. CÓDIGO EN PYTHON

El programa desarrollado en Python se encuentra organizado mediante funciones para validar datos, realizar los cálculos, almacenar los registros, consultar el historial y generar las gráficas.

```

C:\Users > maryo > Desktop > python > analitica_academica.py > ...
1  import math
2  import psycopg
3  import matplotlib.pyplot as plt
4
5  ARCHIVO = "notas.txt"
6
7  # Conexion
8  try:
9      conn = psycopg.connect(
10         dbname="test",
11         host="localhost",
12         port="5432",
13         user="postgres",
14         password="rockyto"
15     )
16 except Exception:
17     print("Error de conexion con PostgreSQL.")
18     exit()
19
20
21 # Validar nota
22 def leer_nota(mensaje):
23     while True:
24         try:
25             nota = float(input(mensaje))
26
27             if 0 <= nota <= 100:
28                 return nota
29
30             print("La nota debe estar entre 0 y 100.")
31
32         except ValueError:
33             print("Ingrese un numero.")
34
35
36 while True:
37
38     # Menu
39     print("\n1. Ingreso de datos")
40     print("2. Historial")
41     print("3. Analitica visual")
42     print("4. Salir")
43
44     opcion = input("Opcion: ")
45
46     # Validar menu
47     if opcion not in ["1", "2", "3", "4"]:
48         print("Opcion no valida.")
```

Figura 23. Código en Python, parte 1 de 5.

```

C:\Users\marjo > Desktop > python > analitica_academica.py > ...
49         continue
50
51     # Opcion 1
52     if opcion == "1":
53
54         carnet = input("Carnet: ")
55         nombre = input("Nombre: ")
56
57         tareas = leer_nota("Tareas (0-100): ")
58         p1 = leer_nota("Parcial 1 (0-100): ")
59         p2 = leer_nota("Parcial 2 (0-100): ")
60         p3 = leer_nota("Parcial 3 (0-100): ")
61         final = leer_nota("Examen final (0-100): ")
62
63         # Calculo
64         nota_exacta = (
65             tareas * 0.25 +
66             p1 * 0.16 +
67             p2 * 0.17 +
68             p3 * 0.17 +
69             final * 0.25
70         )
71
72         # Redondeo
73         entero = math.floor(nota_exacta)
74         decimal = nota_exacta - entero
75
76         if decimal >= 0.5:
77             nota_final = entero + 1
78         else:
79             nota_final = entero
80
81         # Estado
82         if nota_final >= 61:
83             estado = "Aprobado"
84         else:
85             estado = "Reprobado"
86
87         # Resultado
88         print(f"\nNota exacta: {nota_exacta:.2f}")
89         print(f"Nota final: {nota_final}")
90         print(f"Estado: {estado}")
91
92         # Guardar TXT
93         with open(ARCHIVO, "a", encoding="utf-8") as archivo:
94             archivo.write(
95                 f"{carnet}|{nombre}|{tareas}|{p1}|{p2}|"
96                 f"{p3}|{final}|{nota_exacta:.2f}|"

```

Figura 24. Código en Python, parte 2 de 5.

```

C:\Users\marjo > Desktop > python > analitica_academica.py > ...
97         f"{nota_final}|{estado}\n"
98     )
99
100     # Guardar PostgreSQL
101     try:
102         with conn.cursor() as cur:
103             cur.execute(
104                 """
105                 INSERT INTO notas_estudiantes
106                 (carnet,nombre,tareas,p1,p2,p3,
107                  examen_final,nota_exacta,nota_final,estado)
108                 VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)
109                 """
110                 ,
111                 (
112                     carnet,nombre,tareas,p1,p2,p3,
113                     final,nota_exacta,nota_final,estado
114                 )
115             )
116             conn.commit()
117             print("Datos guardados correctamente.")
118
119     except Exception:
120         print("Error en PostgreSQL.")
121         print("Los datos quedaron en notas.txt.")
122         conn.close()
123         break
124
125     # Opcion 2
126     elif opcion == "2":
127
128         print("\n--- HISTORIAL ---")
129
130         with conn.cursor() as cur:
131             cur.execute(
132                 """
133                 SELECT carnet,nombre,nota_final,estado
134                 FROM notas_estudiantes
135                 ORDER BY id
136                 """
137             )
138
139             datos = cur.fetchall()
140
141             if not datos:
142                 print("No hay registros.")
143             else:
144                 print(

```

Figura 25. Código en Python, parte 3 de 5.

```

C: > Users > manyo > Desktop > python > analitica_academica.py > ...
144         print(
145             f"{'CARNE':<15}"
146             f"{'NOMBRE':<25}"
147             f"{'NOTA':<10}"
148             f"{'ESTADO':<15}"
149         )
150
151     for fila in datos:
152         print(
153             f"{fila[0]:<15}"
154             f"{fila[1]:<25}"
155             f"{fila[2]:<10}"
156             f"{fila[3]:<15}"
157         )
158
159 # Opcion 3
160 elif opcion == "3":
161
162     with conn.cursor() as cur:
163         cur.execute(
164             """
165             SELECT nombre,tareas,p1,p2,p3,
166             examen_final,nota_final,estado
167             FROM notas_estudiantes
168             ORDER BY id
169             """
170         )
171
172     datos = cur.fetchall()
173
174     if not datos:
175         print("No hay datos para graficar.")
176         continue
177
178     # Extraer datos
179     nombres = [x[0] for x in datos]
180     tareas = [float(x[1]) for x in datos]
181     p1 = [float(x[2]) for x in datos]
182     p2 = [float(x[3]) for x in datos]
183     p3 = [float(x[4]) for x in datos]
184     finales = [float(x[5]) for x in datos]
185     notas = [int(x[6]) for x in datos]
186     estados = [x[7] for x in datos]
187
188     # Grafica 1
189     plt.figure()
190     plt.bar(nombres, notas)
191     plt.title("Rendimiento individual")

```

Figura 26. Código en Python, parte 4 de 5.

```

... tarea_02.py tarea_02_funciones.py conexion_postgresql.py analitica_academica.py ...
190     plt.bar(nombres, notas)
191     plt.title("Rendimiento individual")
192     plt.xlabel("Estudiantes")
193     plt.ylabel("Nota final")
194     plt.xticks(rotation=45)
195     plt.tight_layout()
196
197     # Grafica 2
198     aprobados = estados.count("Aprobado")
199     reprobados = estados.count("Reprobado")
200
201     plt.figure()
202     plt.pie(
203         [aprobados, reprobados],
204         labels=["Aprobados", "Reprobados"],
205         autopct="%1.1f%%"
206     )
207     plt.title("Indice de aprobacion")
208
209     # Grafica 3
210     promedios = [
211         sum(tareas) / len(tareas),
212         sum(p1) / len(p1),
213         sum(p2) / len(p2),
214         sum(p3) / len(p3),
215         sum(finales) / len(finales)
216     ]
217
218     plt.figure()
219     plt.bar(
220         ["Tareas", "P1", "P2", "P3", "Final"],
221         promedios
222     )
223     plt.title("Desempeno por rubro")
224     plt.ylabel("Promedio")
225     plt.tight_layout()
226
227     plt.show()
228
229     # Salir
230     elif opcion == "4":
231         print("Saliendo...")
232         break
233
234     conn.close()
235
236

```

Figura 27. Código en Python, parte 5 de 5.

XIX. PRUEBAS DE FUNCIONAMIENTO EN PYTHON

Las siguientes capturas muestran la ejecución del programa desarrollado en Python y el funcionamiento de las principales opciones del sistema.

XIX-A. Ingreso de Datos y Resultado

```

=====
INGRESO DE DATOS DEL ESTUDIANTE
=====
Carne: 202202918
Nombre: Maryom
Tareas (0-100): 80
Parcial 1 (0-100): 70
Parcial 2 (0-100): 60
Parcial 3 (0-100): 70
Examen Final (0-100): 60

=====
RESULTADO
=====
Carne: 202202918
Nombre: Maryom
Nota exacta: 68.30
Nota final: 68
Estado: Aprobado

Registro guardado en notas.txt y PostgreSQL.

Presione ENTER para continuar...

```

Figura 28. Ingreso de datos y cálculo de la nota final en Python.

XIX-B. Historial de Estudiantes



Figura 29. Historial de estudiantes mostrado en consola mediante Python.

XIX-C. Análítica Visual

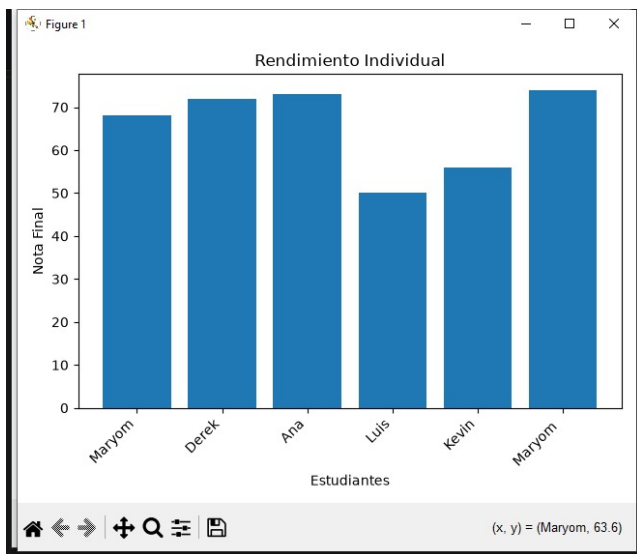


Figura 30. Gráfica de rendimiento individual generada en Python.

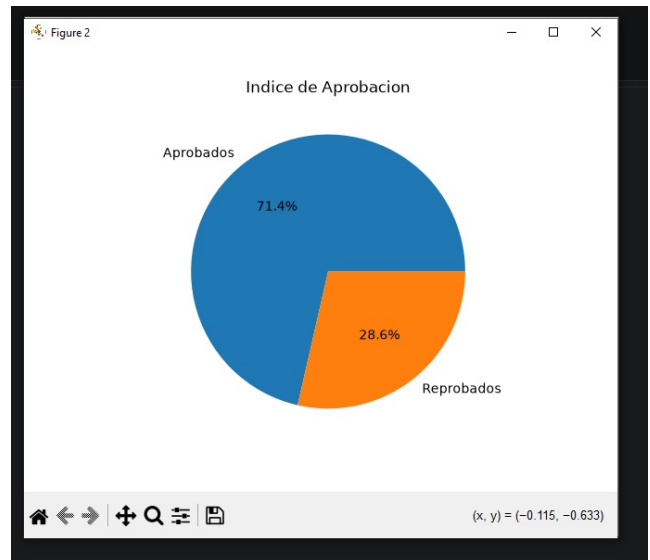


Figura 31. Gráfica del índice de aprobación generada en Python.

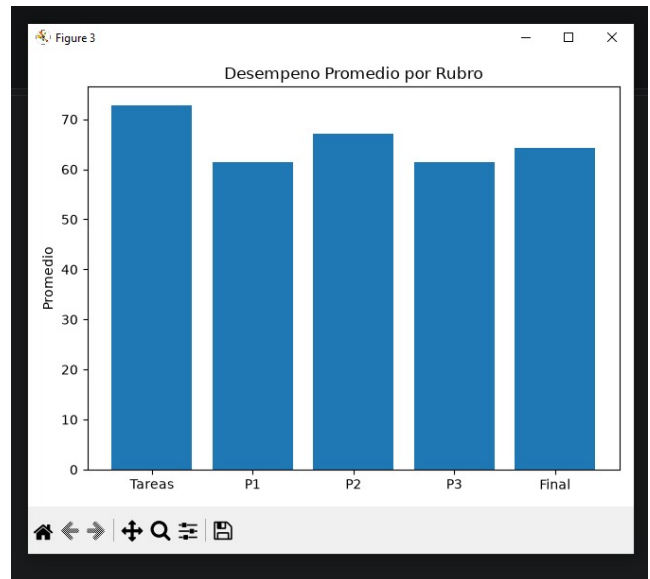


Figura 32. Gráfica del desempeño promedio por rubro generada en Python.

XX. CONCLUSIONES

- Se implementó un sistema para registrar y analizar notas.
- Se utilizaron funciones, PostgreSQL y archivo de texto.
- Las gráficas y validaciones facilitaron el análisis y uso del sistema.